

Logic Synthesis & Verification, Fall 2026

Programming Assignment 1 — Report

Wei-Lung Lin (R15921066)

1 Problem 2: Using ABC

1.1 (a) The two-bit unsigned multiplier mul.blif

Writing $A = (a_1a_0)$ and $B = (b_1b_0)$, the four partial products are grouped by their binary weight:

Weight	Terms in that column
1 (y_0)	a_0b_0
2 (y_1)	a_1b_0, a_0b_1
4 (y_2)	a_1b_1 , plus the carry out of the weight-2 column
8 (y_3)	carry out of the weight-4 column

With $t_0 = a_1b_0$, $t_1 = a_0b_1$ and $t_2 = a_1b_1$ this gives $y_0 = a_0b_0$, $y_1 = t_0 \oplus t_1$, $y_2 = t_2 \oplus (t_0t_1)$ and $y_3 = t_0t_1t_2$. Each equation is written as one `.names` table:

```
.model mul
.inputs a0 a1 b0 b1
.outputs y0 y1 y2 y3
.names a0 b0 y0
11 1
.names a1 b0 t0
11 1
.names a0 b1 t1
11 1
.names a1 b1 t2
11 1

.names t0 t1 y1
01 1
10 1
.names t0 t1 t2 y2
001 1
011 1
101 1
110 1
.names t0 t1 t2 y3
111 1

.end
```

The circuit was verified against a reference network built directly from the exhaustive truth table; ABC's `cec` reports *Networks are equivalent*.

1.2 (b) Running the seven steps in ABC

Steps 1–2: read and print_stats. The network has 4 inputs, 4 outputs, 7 logic nodes, 16 edges, 11 cubes and 2 levels.

```
abc 01> read ./lsv/pa1/mul.blif
abc 02> print_stats
mul          : i/o =   4/   4  lat =   0  nd =   7  edge =   16  cube =   11  lev = 2
```

Figure 1: Reading `mul.blif` and printing its statistics.

Step 3: show — the logic network. Coral triangles are the primary inputs (bottom) and primary outputs (top); each ellipse is a logic node labelled with its node ID and its SOP cube table.

Network structure visualized by ABC
Benchmark "mul". Time was Fri Sep 25 14:39:40 2026.

The network contains 7 logic nodes and 0 latches.

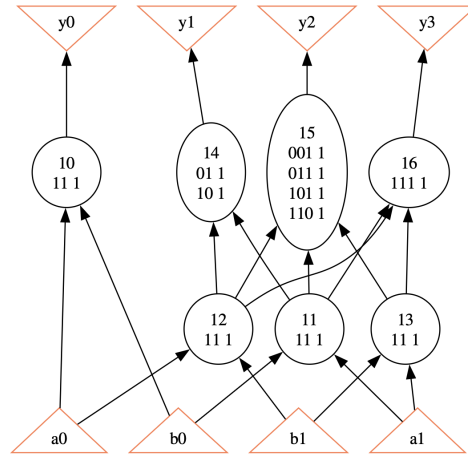


Figure 2: Original SOP logic network (7 nodes).

Steps 4–5: strash and show — the AIG. Every node is now a two-input AND, so the ellipses carry only an ID and no cube table. Inversions are not separate nodes: they are recorded on the edges and drawn *dotted*. The 7 SOP nodes expand into 15 AND nodes and the depth grows from 2 to 5.

Network structure visualized by ABC
Benchmark "mul". Time was Fri Sep 25 14:40:57 2026.

The network contains 15 logic nodes and 0 latches.

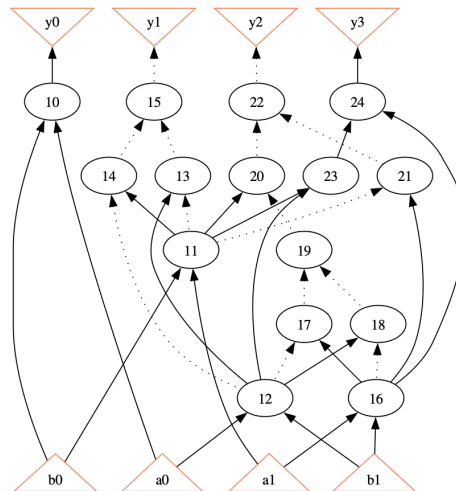
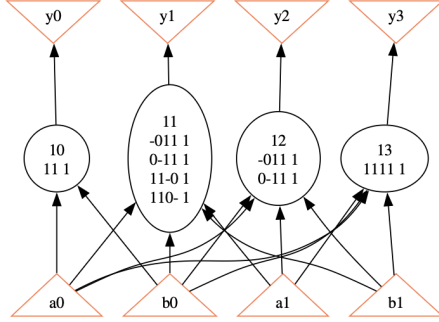


Figure 3: Structurally hashed AIG (15 AND nodes; dotted edges are complemented).

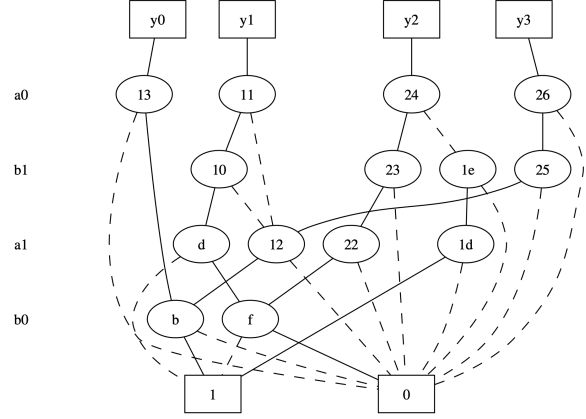
Steps 6–7: collapse and show_bdd -g — the BDD. After collapsing, the network contains exactly 4 nodes, one per primary output, and each of them is driven directly by the primary inputs.

Network structure visualized by ABC
Benchmark "mul". Time was Fri Sep 25 14:41:27 2026.

The network contains 4 logic nodes and 0 latches.



(a) Network after collapse (4 nodes).



(b) Global BDDs shown by `show_bdd -g`.

Figure 4: Collapsed network and its global BDDs.

In the BDD picture the variable order is printed down the left-hand side, each row holds the decision nodes of one variable, a *solid* edge is the then-branch ($v = 1$), a *dashed* edge is the else-branch ($v = 0$), and the two boxes at the bottom are the constant terminals. All four outputs share one graph, so isomorphic sub-graphs are drawn only once.

2 Problem 3: ABC Boolean Function Representations

2.1 (a).1 Logic network in AIG (aig) vs. structurally hashed AIG (strash)

	nd	edge	node function	size	lev
original (SOP)	7	16	SOP	cube = 11	2
after aig	7	16	AIG	aig = 15	2
after strash	—	—	2-input AND	and = 15	5

```

abc 04> read ./lsv/pa1/mul.blif
abc 05> print_stats
mul          : i/o = 4/ 4 lat = 0 nd = 7 edge = 16 cube = 11 lev = 2
abc 05> aig
abc 05> print_stats
mul          : i/o = 4/ 4 lat = 0 nd = 7 edge = 16 aig = 15 lev = 2
abc 05> show

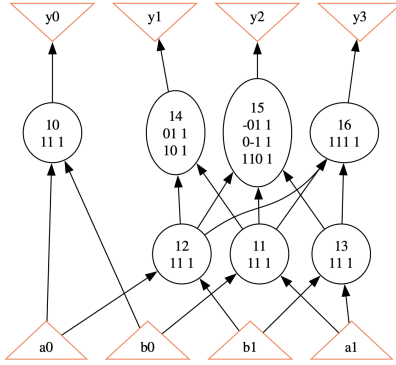
abc 05> read ./lsv/pa1/mul.blif
abc 06> print_stats
mul          : i/o = 4/ 4 lat = 0 nd = 7 edge = 16 cube = 11 lev = 2
abc 06> strash
abc 07> print_stats
mul          : i/o = 4/ 4 lat = 0 and = 15 lev = 5
abc 07> show

```

Figure 5: Statistics before and after aig (top) and strash (bottom).

Network structure visualized by ABC
Benchmark "mul". Time was Fri Sep 25 14:57:04 2026.

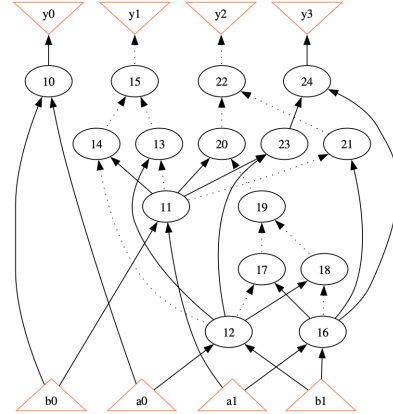
The network contains 7 logic nodes and 0 latches.



(a) aig: 7 nodes, unchanged structure.

Network structure visualized by ABC
Benchmark "mul". Time was Fri Sep 25 14:57:50 2026.

The network contains 15 logic nodes and 0 latches.



(b) strash: 15 AND nodes, dotted edges.

Figure 6: show after aig and after strash.

Findings.

- **aig** changes only *how each node stores its local function*. The network remains a logic network: the node count (7), the edge count (16) and the depth (2) are all unchanged, and the drawn structure is identical to the original. Only the reported size switches from **cube** to **aig**. (The cube list printed inside node 15 also becomes shorter, because **show** re-derives a cover from the stored AIG when it draws the node.)
- **strash** rebuilds the *whole* network as one global AIG. The node boundaries disappear — **print_stats** no longer reports **nd/edge** at all — every node becomes a two-input AND, and inverters are replaced by complemented edges. The depth grows from 2 to 5 because the

multi-level structure that used to be hidden inside a single SOP node is now flattened into the global graph.

- While building the AIG, **strash** performs *structural hashing*: before a new AND node is created the hash table is consulted, so two structurally identical nodes are merged into one. This is the essential difference from **aig**, which never merges anything. On this small multiplier both happen to reach 15 nodes, but on a network containing two identical sub-circuits **aig** keeps both copies while **strash** collapses them into a single node.

2.2 (a).2 Logic network in BDD (bdd) vs. collapsed BDD (collapse)

	nd	edge	bdd	lev	BDD variables are
after bdd	7	16	16	2	the node's own fanins
after collapse	4	14	16	1	the primary inputs

```

abc 07> read ./lsv/pa1/mul.blif
abc 08> print_stats
mul          : i/o = 4/ 4 lat = 0 nd = 7 edge = 16 cube = 11 lev = 2
abc 08> bdd
abc 08> print_stats
mul          : i/o = 4/ 4 lat = 0 nd = 7 edge = 16 bdd = 16 lev = 2
abc 08> show

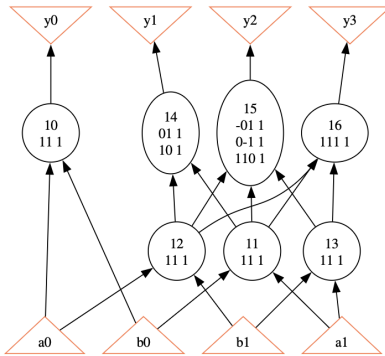
abc 08> read ./lsv/pa1/mul.blif
abc 09> print_stats
mul          : i/o = 4/ 4 lat = 0 nd = 7 edge = 16 cube = 11 lev = 2
abc 09> collapse
abc 10> print_stats
mul          : i/o = 4/ 4 lat = 0 nd = 4 edge = 14 bdd = 16 lev = 1
abc 10> show

```

Figure 7: Statistics before and after bdd (top) and collapse (bottom).

Network structure visualized by ABC
Benchmark "mul". Time was Fri Sep 25 15:02:04 2026.

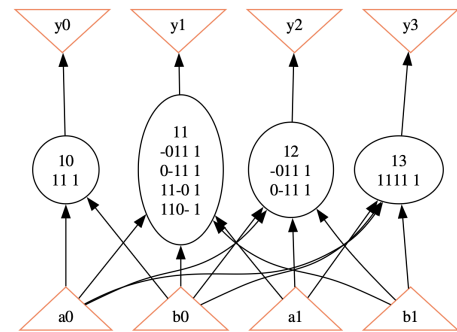
The network contains 7 logic nodes and 0 latches.



(a) bdd: 7 nodes, structure unchanged.

Network structure visualized by ABC
Benchmark "mul". Time was Fri Sep 25 15:03:03 2026.

The network contains 4 logic nodes and 0 latches.



(b) collapse: 4 nodes, one per output.

Figure 8: show after bdd and after collapse.

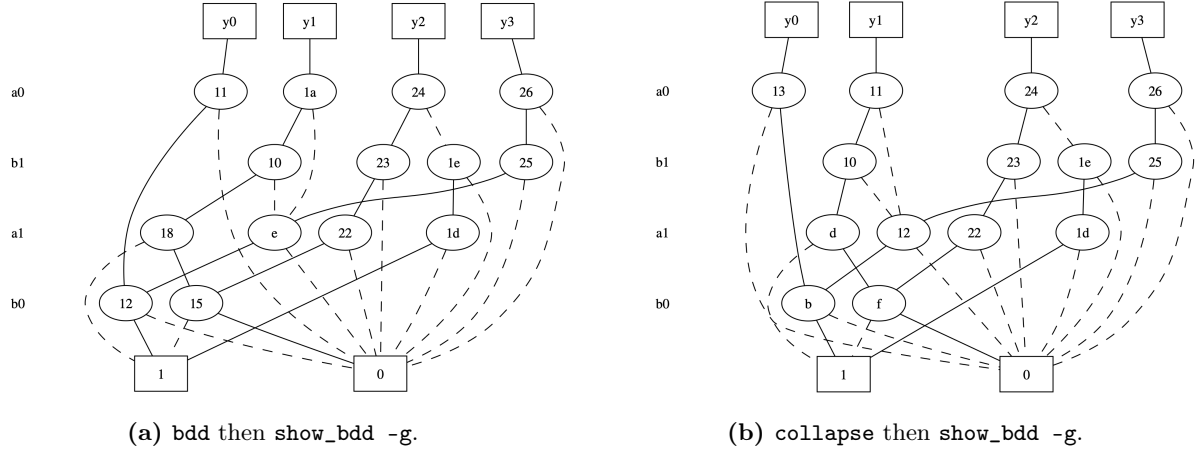


Figure 9: Global BDDs of the four primary outputs.

Findings.

- This pair mirrors the previous one: **bdd** is a *per-node* conversion, **collapse** is a *global* one.
- **bdd** stores each node's local function as a BDD whose variables are that node's own fanins, which may be internal signals. The network keeps all 7 nodes, 16 edges and 2 levels.
- **collapse** builds one global BDD per primary output over the primary inputs. All internal signals are substituted away, so the network shrinks to 4 nodes (one per output) with depth 1, and every node is fed directly by primary inputs. Writing the result out as SOP confirms this: after **bdd** the cover of y_2 is over t_0 t_1 t_2 , whereas after **collapse** it is over a_0 a_1 b_0 b_1 .
- The variable order in the BDD figures is not the declaration order, because dynamic variable reordering is enabled by default. Since the size of a global BDD is highly sensitive to this order — and grows exponentially in the worst case — **collapse** is practical only for small circuits, which is why ABC uses the AIG as its working representation.

2.3 (b) From a structurally hashed AIG to a logic network with SOP node functions

Command sequence. The following four commands do the job:

```
read lsv/pa1/mul.blif
strash
logic
sop
```

logic converts the strashed AIG back into a logic network (its help text reads “*transforms an AIG into a logic network with SOPs*”), and **sop** makes sure every node function is stored in sum-of-products form.

```
abc 16> read ./lsv/pa1/mul.blif
abc 17> strash
abc 18> logic
abc 19> sop
abc 19> show
abc 19> write_blif /dev/stdout
# Benchmark "mul" written by ABC on Fri Sep 25 15:16:12 2026
```

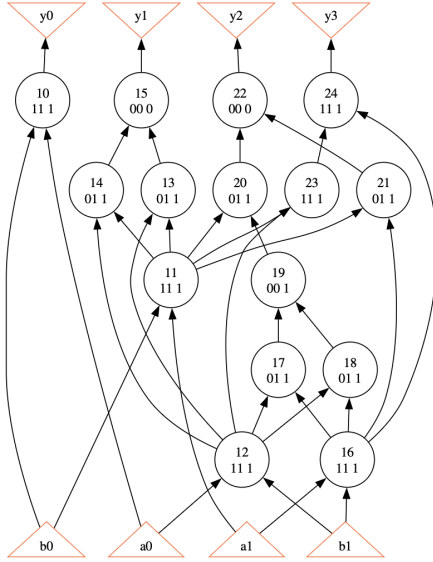
Figure 10: The command sequence applied to the two-bit multiplier.

The conversion is confirmed by **print_stats**, which changes from **and** = 15, **lev** = 5 (a strashed AIG) to **nd** = 15, **edge** = 30, **cube** = 15, **lev** = 5 (a logic network). Each of the 15 AND nodes

becomes one logic node carrying a single cube; in the drawing all edges are solid, since a logic network has no complemented edges.

Network structure visualized by ABC
Benchmark "mul". Time was Fri Sep 25 15:16:09 2026.

The network contains 15 logic nodes and 0 latches.



(a) `show`: 15 logic nodes with cube tables.

```
abc 19> write_blif /dev/stdout
# Benchmark "mul" written by ABC on Fri Sep 25 15:16:12 2026
.model mul
.inputs a0 a1 b0 b1
.outputs y0 y1 y2 y3
.names a0 b0 y0
11 1
.names a1 b0 new_n10
11 1
.names a0 b1 new_n11
11 1
.names new_n10 new_n11 new_n12
01 1
.names new_n11 new_n10 new_n13
01 1
.names new_n12 new_n13 y1
00 0
.names a1 b1 new_n15
11 1
.names new_n11 new_n15 new_n16
01 1
.names new_n15 new_n11 new_n17
01 1
.names new_n16 new_n17 new_n18
00 1
.names new_n18 new_n10 new_n19
01 1
.names new_n10 new_n15 new_n20
01 1
.names new_n19 new_n20 y2
00 0
.names new_n10 new_n11 new_n22
11 1
.names new_n15 new_n22 y3
11 1
.end
```

(b) `write_blif` output: SOP node functions.

Figure 11: Resulting SOP logic network.

Running `cec` between the result and the original `mul.blif` reports that the networks are equivalent, so the command sequence is functionality-preserving.